Graph drawing by stress minimization

Laslo Hauschild, Lennart Peters

March 30, 2026

Introduction

Introduction

Graphs are a prominent object in modern mathematics, useful for abstracting complex reallife situations.

- Railway networks, data structures, geometric objects
- Model relationships: vertices for entities, edges for connections

Example: Two companies with employees as vertices and daily interactions as edges.

- Compare communication hierarchy and department interactions
- Difficulty: meticulously searching through huge lists of vertices and edges

Goal: Represent finite graphs in $\mathbb{R}^2$ and $\mathbb{R}^3$ esthetically.

- Drawing: vertices as points, edges as straight-line segments
- Quickly understand structure and type of a given graph

Graph Embedding

Graph embedding

Graph definition:

- set of vertices V, set of edges $E \subseteq \{\{u, v\} \mid u, v \in E\}$ then $G = (V, E)$

Our goal is to draw graphs, for our purposes a drawable graph has a finite set of vertices.

- finite $V \implies$ finite E

A graph embedding p is an injective map $p : V \to A$, which maps every $v \in V$. p induces an embedding of E, therefore p is a lossless embedding of G in A.

- $\exists p \iff |V| \leq |A|$

Let us call $H = p(G)$ and choose $A = \mathbb{R}^n, \quad n = 2, 3$

Modeling stress

We add a drawing function $d : F \to \mathbb{R}, \{a, b\} \mapsto \overline{ab}$, giving us a drawing of H.

$W \subseteq \mathbb{R}^n$ essentially randomly chosen, drawings of H are mostly tangled. We use a stress function S that assigns the graph H a real number, the stress in H. Minima of S are our desired representations of G, if possible the global minimum as ideal.

FR model

developed 1991, by Fruchterman and Reingold, following two main principles [4]:

- drawing neighboring vertices near each other
- vertices should not be drawn too near to each other

The FR model uses the following stress-function:

$$S_{FR}(p(G)) = \frac{1}{6} \sum_{\{u,v\} \in E} \| p(u) - p(v) \|^3 - \sum_{\{u,v\} \subseteq V} \log \| p(u) - p(v) \| \tag{1}$$

Observe:

- the further apart neighboring vertices get mapped the bigger the stress
- small distances between vertices let the logarithm explode
- the logarithm dominates at small distances and hinders the graph from collapsing, or converging into a single point

S_{FR} satisfies the two main principles, while being rather quick when implemented into an algorithm.

ERLL model

The Edge Repulsion LinLog model by Andreas Noack, 2003, places its focus on the graph's edges. For $v \in V$, let $\deg(v)$ denote the number of edges v is an endpoint of [1].

$$\begin{aligned} S_{ERLL}(p(G)) = & \frac{1}{2} \sum_{\{u,v\} \in E} \| p(u) - p(v) \| \\ & - \sum_{\{u,v\} \subseteq V} \deg(u) \deg(v) \log \| p(u) - p(v) \| \end{aligned} \tag{2}$$

Observe:

- edges cause both attraction and repulsion
- G of uniform degrees: FR $\approx$ ERLL scaled with $c = \deg(v)^2$
- G of non-uniform degrees: FR repulsion remains the same, but vertices u, v repel each other proportional to $\deg(u)\deg(v)$ in ERLL.

$\implies$ ERLL spreads out "big nodes" compared to FR, usefull in highly non-uniform graphs

Implementation

Plotting a Graph Embedding

```julia
using Plots
using Graphs

function plot_graph(g::SimpleGraph, p::Matrix{Float64})
    plt = plot(legend = false, aspect_ratio = :equal)
    for e in edges(g)
        plot!(plt, [p[1, src(e)], p[1, dst(e)]],
            [p[2, src(e)], p[2, dst(e)]], color = :black)
    end
    scatter!(plt, p[1, :], p[2, :], markersize=5, color = :blue)
    return plt
end
```

- Dependencies: Graphs [5] and Plots [6] libraries
- Arguments: A graph g and a position matrix p with vertex coordinates
- Order of plotting matters: edges first, then vertices

Plotting a Graph Embedding in 3 Dimensions

```julia
function plot_graph_3d(g::SimpleGraph, p::Matrix{Float64})
    plt = plot(legend = false, aspect_ratio = :equal)
    for e in edges(g)
        plot!(plt, [p[1, src(e)], p[1, dst(e)]],
            [p[2, src(e)], p[2, dst(e)]],
            [p[3, src(e)], p[3, dst(e)]], color = :black)
    end
    scatter!(plt, p[1, :], p[2, :], p[3, :],
        markersize=5, color = :blue)
    return plt
end
```

Stress functions

- ERLL Stress and FR Stress are structured similarly
  - edge stress
  - vertex stress
  - combine function
- Idea: One function that takes anonymous functions

Stress Function

```
function stress(g::SimpleGraph, edge_stress_f::Function,
    vertex_stress_f::Function, combine_f::Function)

    n = nv(g)
    edge_stress = 0
    for e in edges(g)
        edge_stress += edge_stress_f(src(e), dst(e))
    end
    vertex_stress = 0
    for i in 1:n
        for j in (i+1):n
            vertex_stress += vertex_stress_f(i, j)
        end
    end
    return combine_f(edge_stress, vertex_stress)
end
```

Stress Function

```julia
function fr_stress(g::SimpleGraph, p::Matrix{Float64})
    D = pairwise(Euclidean(), p, dims=2)
    edge_stress_f = (i, j) -> D[i, j]^3
    vertex_stress_f = (i, j) -> log(D[i, j] + 10^(-6))
    combine_f = (x, y) -> x / 6 - y
    return stress(g, edge_stress_f, vertex_stress_f, combine_f)
end

function erll_stress(g::SimpleGraph, p::Matrix{Float64}, deg::Vector{Int})
    D = pairwise(Euclidean(), p, dims=2)
    edge_stress_f = (i, j) -> D[i, j]
    vertex_stress_f = (i, j) -> deg[i] * deg[j] * log(D[i, j] + 10^(-6))
    combine_f = (x, y) -> x / 2 - y
    return stress(g, edge_stress_f, vertex_stress_f, combine_f)
end
```

- We do $\log(D[i,j] + 10^{-6})$ to limit the size of the logarithm
- For efficiency, `pairwise` function (Distances library [2]) to compute the distance matrix D.
- ERLL stress has additional argument: degree of each vertex (why: could calculate in stress function, this would be calculated every time, which is inefficient)

Gradient Descent

Zygote

- We do not derive the gradients manually
- Zygote library [3] to compute the gradients automatically
- Uses source-to-source automatic differentiation:
  1. Transforms code into differentiable form
  2. Applies the chain rule
  3. Done 🎉

Gradient Descent

Principle of gradient descent:

1. Initialize vertex positions randomly in 2D space
2. Compute stress and its gradient at each step
3. Update positions minimally in the direction of steepest descent
4. Check for convergence: stop after max steps or if stress values do not change meaningfully
   - $\rightarrow$ Do not check for equality, check if relative difference is smaller than a small threshold

```julia
function simple_gradient_descent(g::SimpleGraph, stress_fn::Function,
    alpha::Float64, max_steps::Int64, ...)

    n = nv(g) # Number of vertices
    p = rand(2, n) # Initial positions
    stresses = Float64[]

    for k in 1:max_steps
        s, (grad,) = Zygote.withgradient(q -> stress_fn(g, q), p)
        push!(stresses, s)

        if length(stresses) > 1 &&
            abs((stresses[end] - stresses[end-1]) / stresses[end-1]) < 1e-7
            ... # result plotting code
            break
        end

        p -= alpha * grad

        ... # continuous plotting code
    end
end
```

Break condition

Definitons

- $s_l :=$ last stress value
- $s_{l-1} :=$ second to last stress value

Convergence criterion

Natural way (absolute difference): $\| s_l - s_{l-1} \| < \epsilon$.

$\rightarrow$ Problem: Depends on absolute size of stress values.

Better way (relative difference): $\| \frac{s_l - s_{l-1}}{s_{l-1}} \| < \epsilon$.

Nesterov's accelerated gradient descent

Motivation: Simple gradient descent can be slow, especially in areas where the stress function is flat.

Idea: A ball rolling downhill

- Has inertia
- Gains speed as it goes down the hill
- Can escape local minima and find better solutions

Implementation:

- In addition to current gradient, we also consider previous update (momentum) when updating positions
- Friction modelled by $\beta = 0.9$, meaning we keep 90% of previous momentum

```julia
function nesterov_gradient_descent(g::SimpleGraph, stress_fn::Function, alpha::
    beta::Float64, max_steps::Int64, ...)

    n = nv(g) # Number of vertices
    p = rand(2, n) # Initial positions
    stresses = Float64[]
    momentum = zeros(2, n)
    for k in 0:max_steps
        s, (grad,) = Zygote.withgradient(q -> stress_fn(g, q), p + beta * momen
        new_momentum = beta * momentum - alpha * grad
        push!(stresses, s)
        if length(stresses) > 50 &&
            abs((stresses[end] - stresses[end-1]) / stresses[end-1]) < 1e-7
            ... # result plotting code
            break
        end
        p += new_momentum
        momentum = new_momentum
        ... # continuous plotting code
    end
end
```

Results

Good convergence

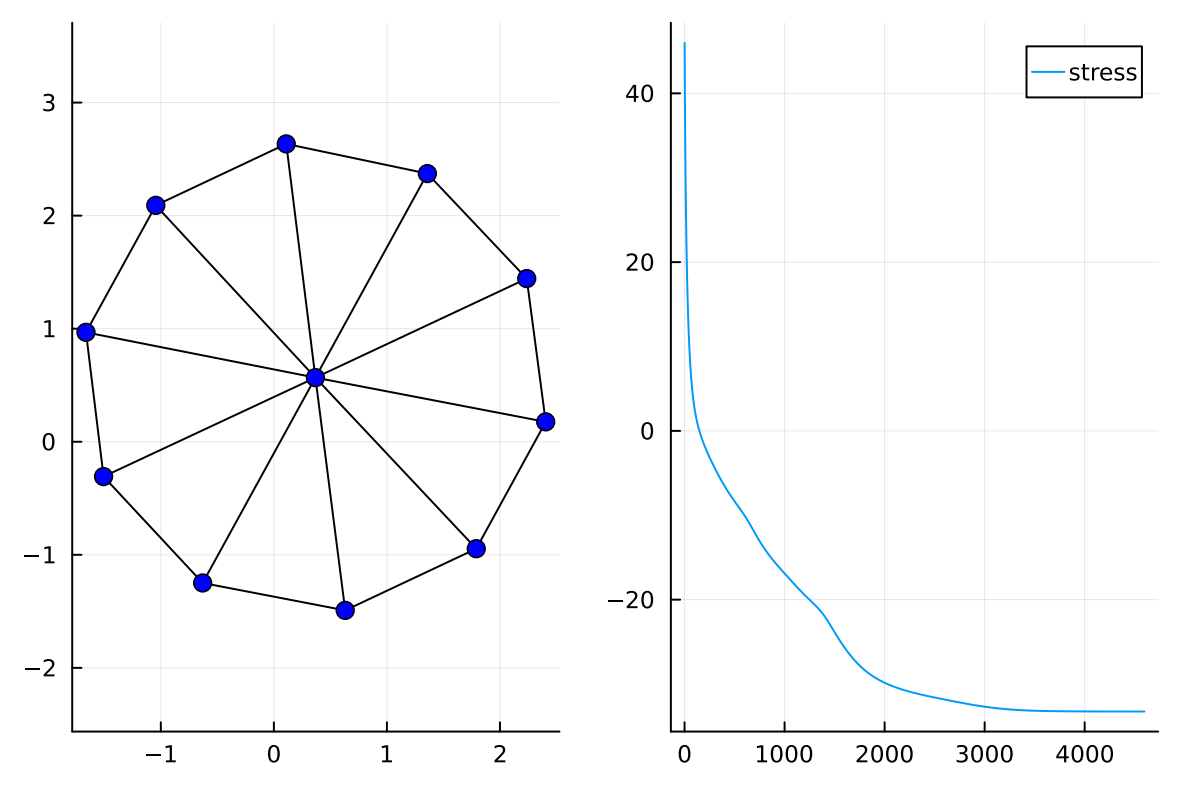

Figure 1: Wheel Graph 11, Simple Gradient Descent, $\alpha = 0.001$, Took 4597 Iterations

Bad convergence

Less than 10% of the runs, the graphs converge poorly:

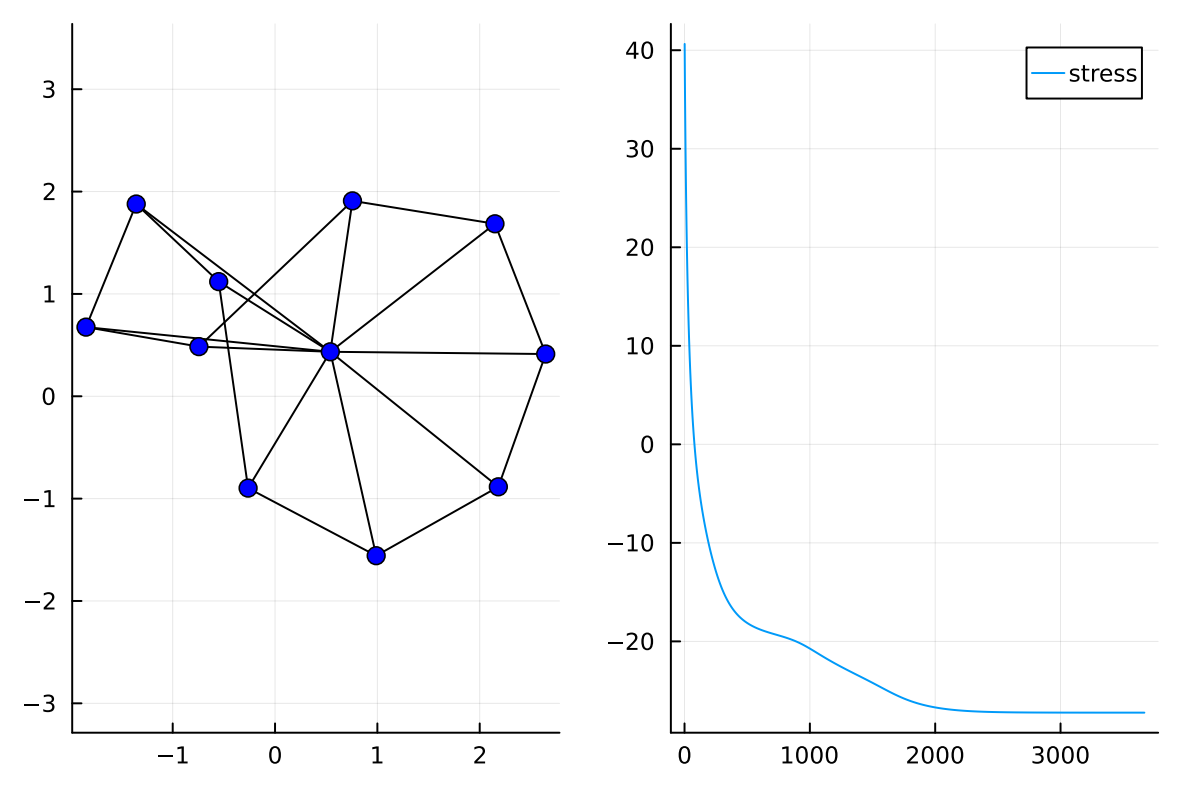

Figure 2: Wheel Graph 11, Simple Gradient Descent, $\alpha = 0.001$, Took 3669 Iterations

Figuring out good learning rates

We had problems with the suggested learning rates.

- FR stress: learning rates were fine
- ERLL: learning rates were too small, making convergence very slow
- We try different learning rates and see how they affect convergence speed and quality of the results

Learning rate $\alpha = 0.01$

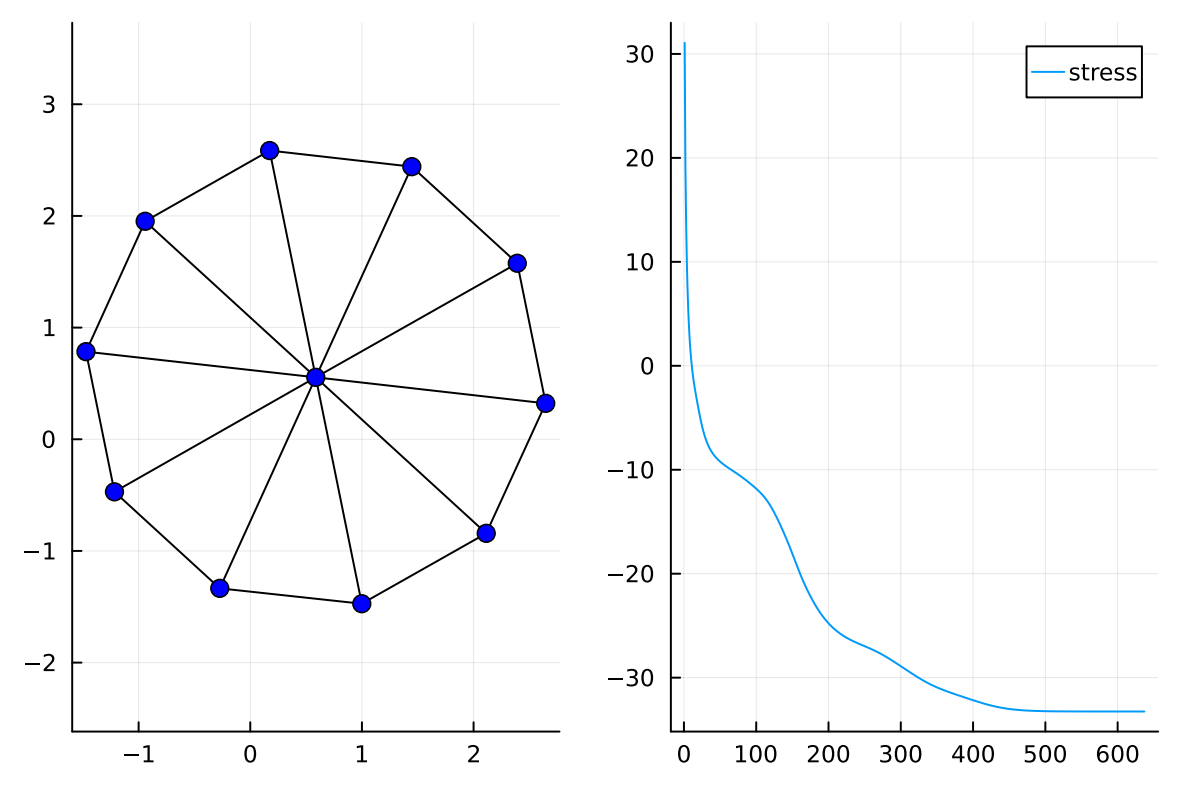

Figure 3: Wheel Graph 11, Simple Gradient Descent, $\alpha = 0.01$, Took 428 Iterations

Learning rate $\alpha = 0.1$

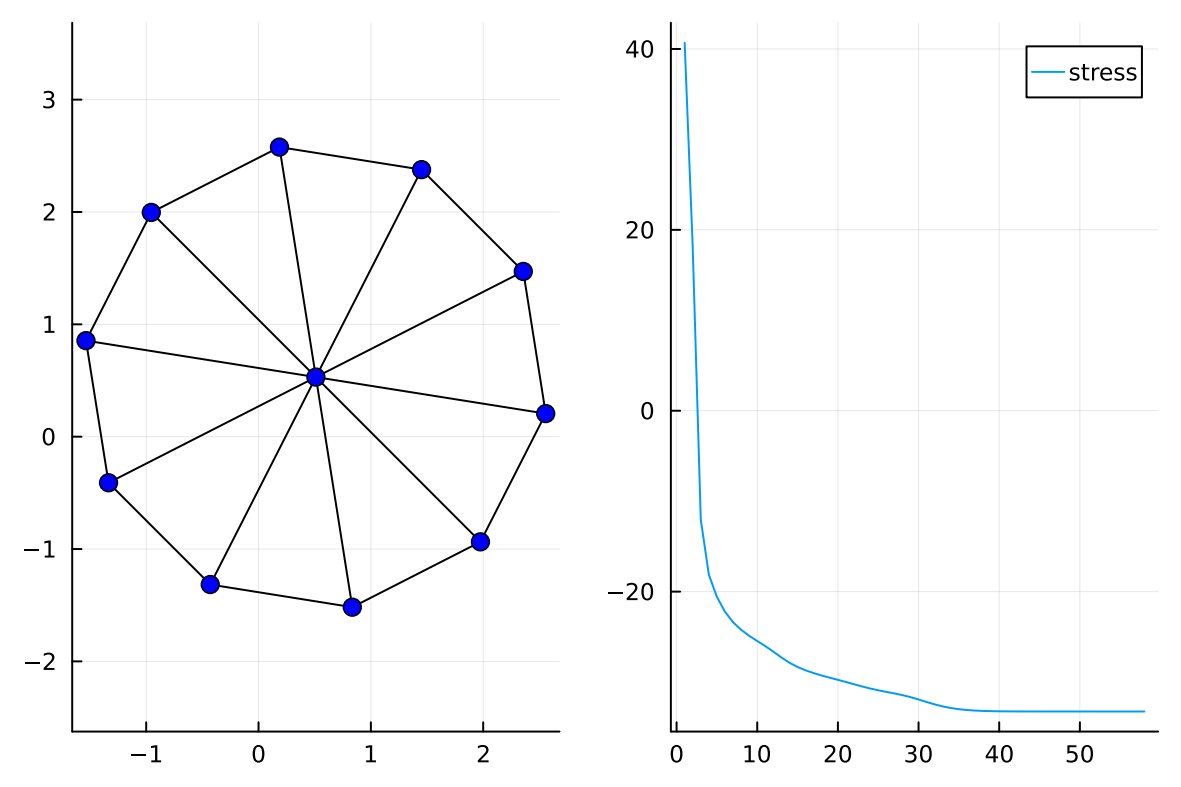

Figure 4: Wheel Graph 11, Simple Gradient Descent, $\alpha = 0.1$, Took 57 Iterations

Learning rate $\alpha = 1.0$

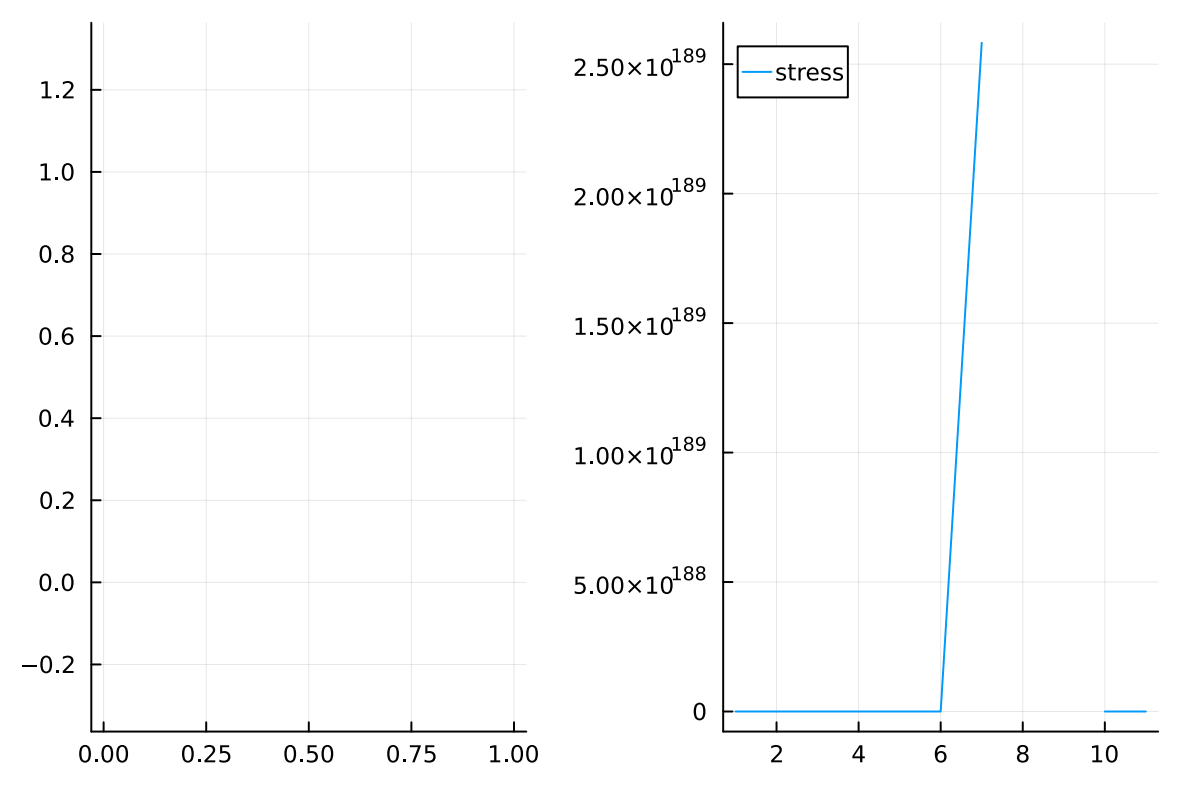

Figure 5: Wheel Graph 11, Simple Gradient Descent, $\alpha = 1.0$, Took 11 Iterations

Increasing the learning rate. . .

- makes convergence faster
- increases the rate of poorly converged graphs (difficult to quantify)
- makes the stress go up at bigger values
- will prevent convergence at very big values because of number overflow

With simple gradient descent and ERLL and $\alpha = 0.001$, convergence is painfully slow. Increasing it to $\alpha = 0.1$ works:

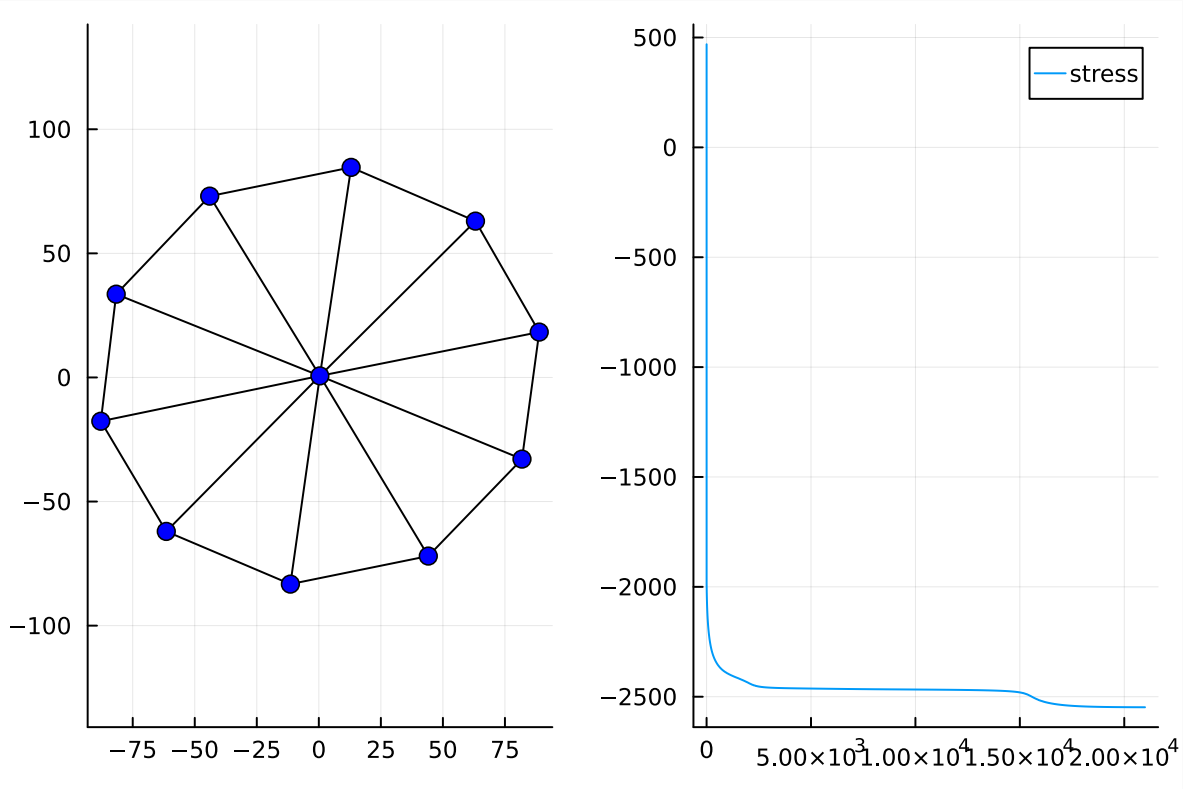

Figure 6: Wheel Graph 11, Simple Gradient Descent, $\alpha = 0.1$, Took 20994 Iterations

A good learning rate

FR Stress $\alpha = 0.001$ offers reasonable convergence speed and good results.

ERLL stress $\alpha = 0.001$ is too small, making convergence very slow. Increasing it to $\alpha = 0.1$ works.

We will use these learning rates for the rest of the project.

Icosahedral – ERLL

Nesterov Gradient Descent, ERLL, $\alpha = 0.1$, $\epsilon = 1e - 7$

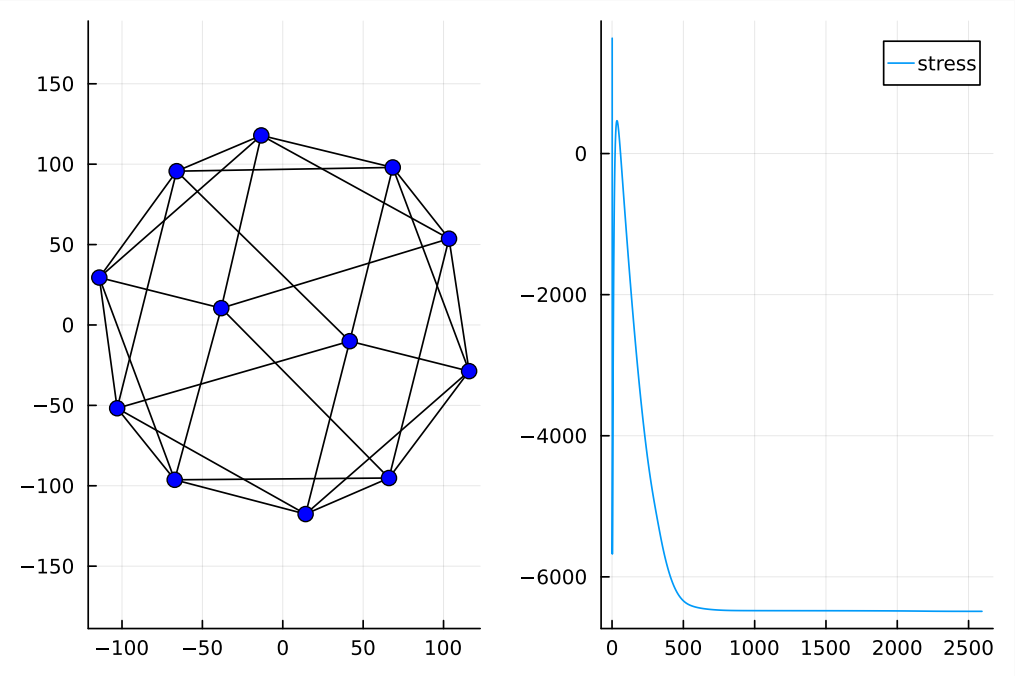

Figure 7: Icosahedral Graph, Took 2593 Iterations

Icosahedral – FR

Nesterov Gradient Descent, FR, $\alpha = 0.001$, $\epsilon = 1e - 7$

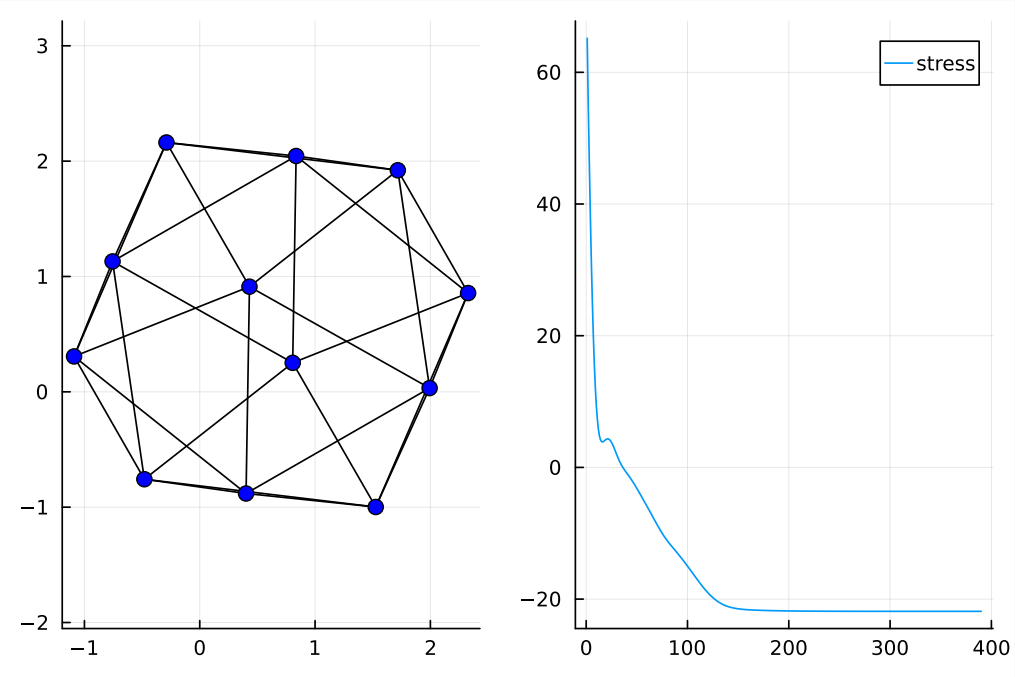

Figure 8: Icosahedral Graph, Took 389 Iterations

Karate Network – ERLL

Nesterov Gradient Descent, ERLL, $\alpha = 0.1$, $\epsilon = 1e - 7$

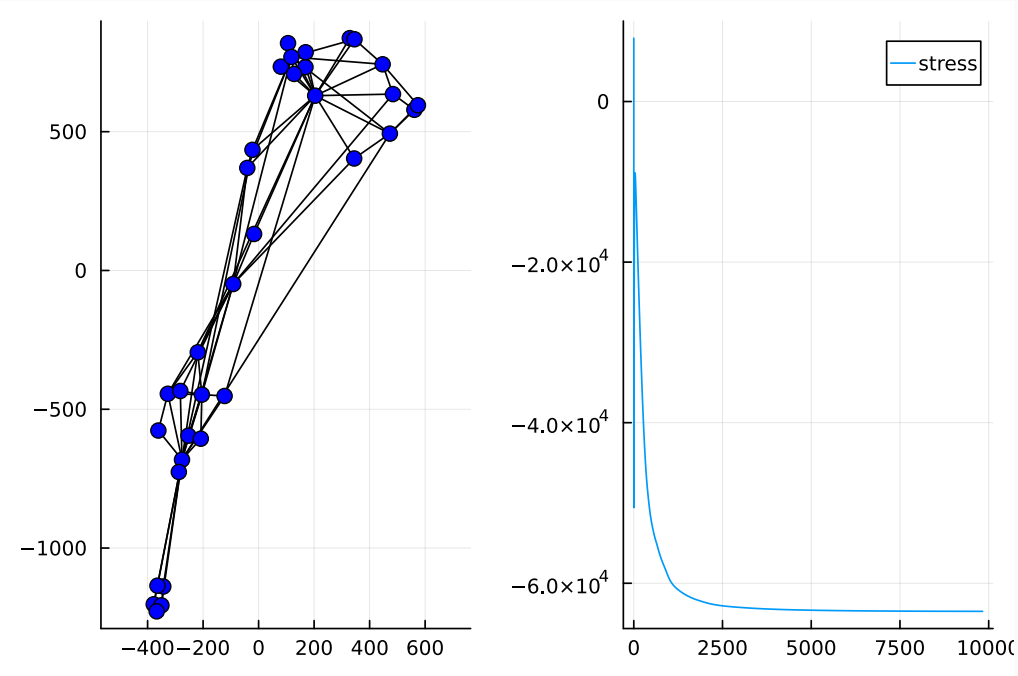

Figure 9: Karate Network, Took 9828 Iterations

Karate Network – FR

Nesterov Gradient Descent, FR, $\alpha = 0.001$, $\epsilon = 1e - 7$

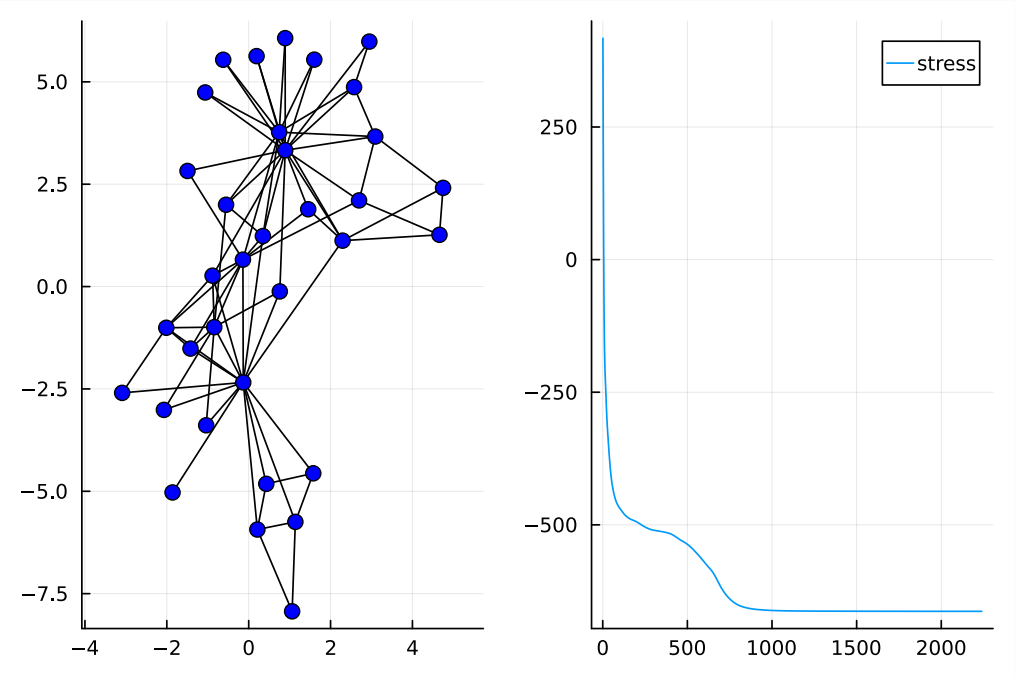

Figure 10: Karate Network, Took 2238 Iterations

Dorogovtsev Mendes Graph – ERLL

Nesterov Gradient Descent, ERLL, $\alpha = 0.1$, $\epsilon = 1e - 7$

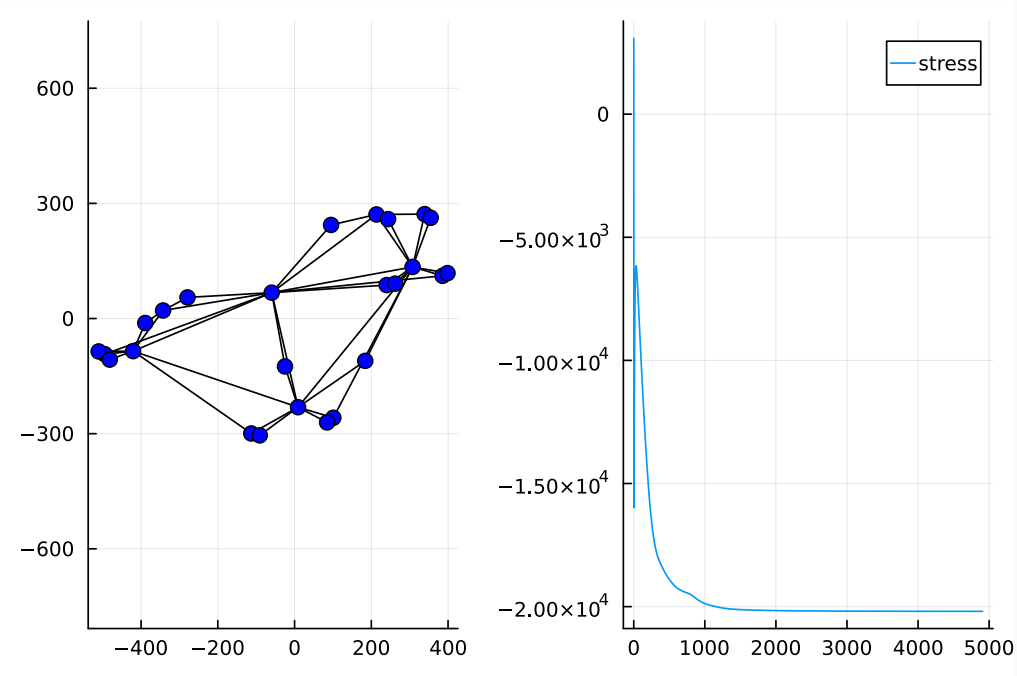

Figure 11: Dorogovtsev Mendes Graph, Took 4907 Iterations

Dorogovtsev Mendes Graph – FR

Nesterov Gradient Descent, FR, $\alpha = 0.001$, $\epsilon = 1e - 7$

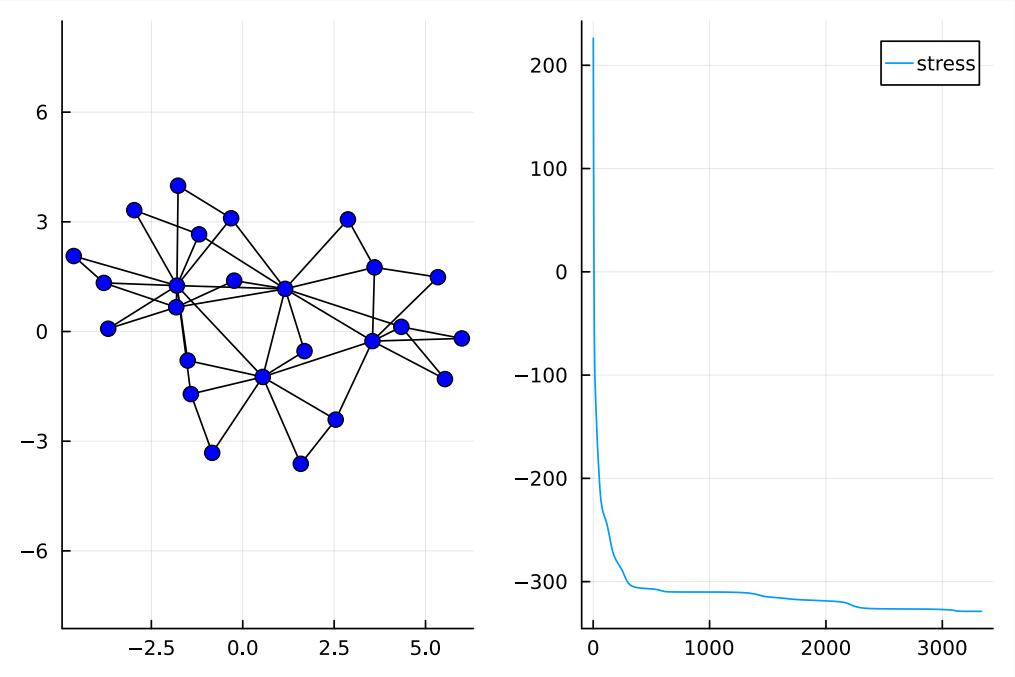

Figure 12: Dorogovtsev Mendes Graph, Took 3335 Iterations

Circular Ladder Graph – ERLL

Nesterov Gradient Descent, ERLL, $\alpha = 0.1$, $\epsilon = 1e - 7$

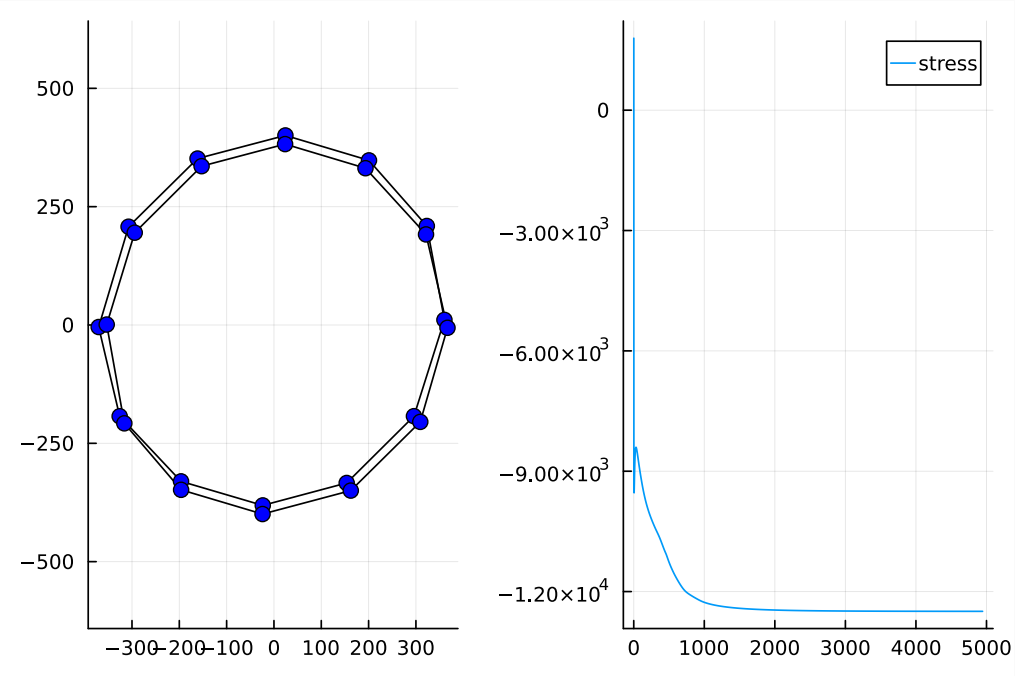

Figure 13: Circular Ladder Graph, Took 4944 Iterations

Circular Ladder Graph – FR

Nesterov Gradient Descent, FR, $\alpha = 0.001$, $\epsilon = 1e - 7$

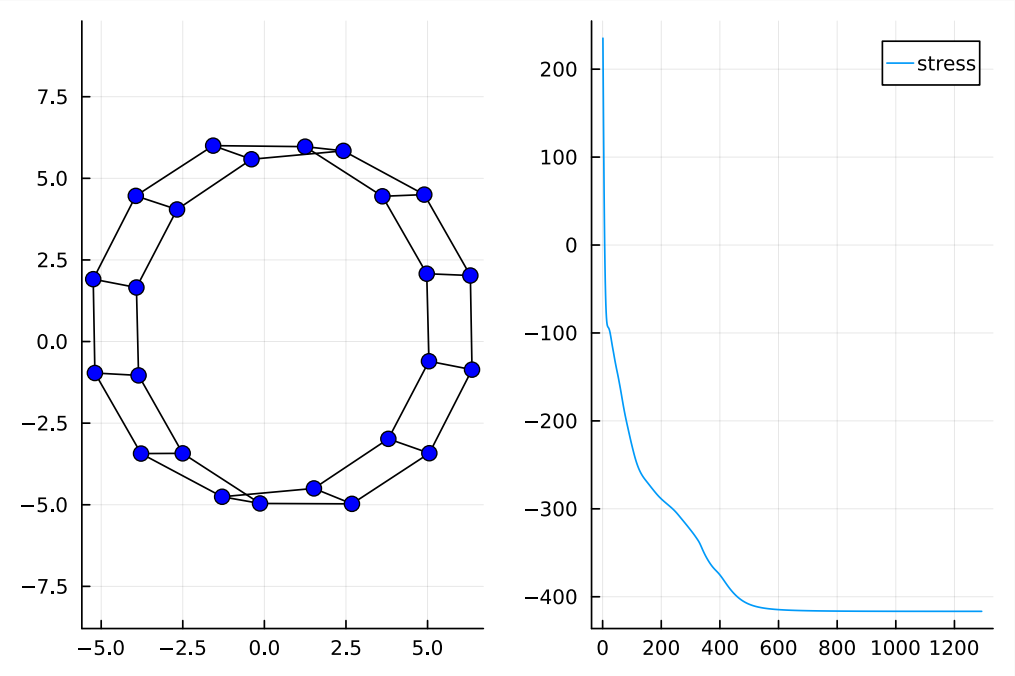

Figure 14: Circular Ladder Graph, Took 1292 Iterations

Watts Strogatz Graph – ERLL

Nesterov Gradient Descent, ERLL, $\alpha = 0.1$, $\epsilon = 1e - 7$

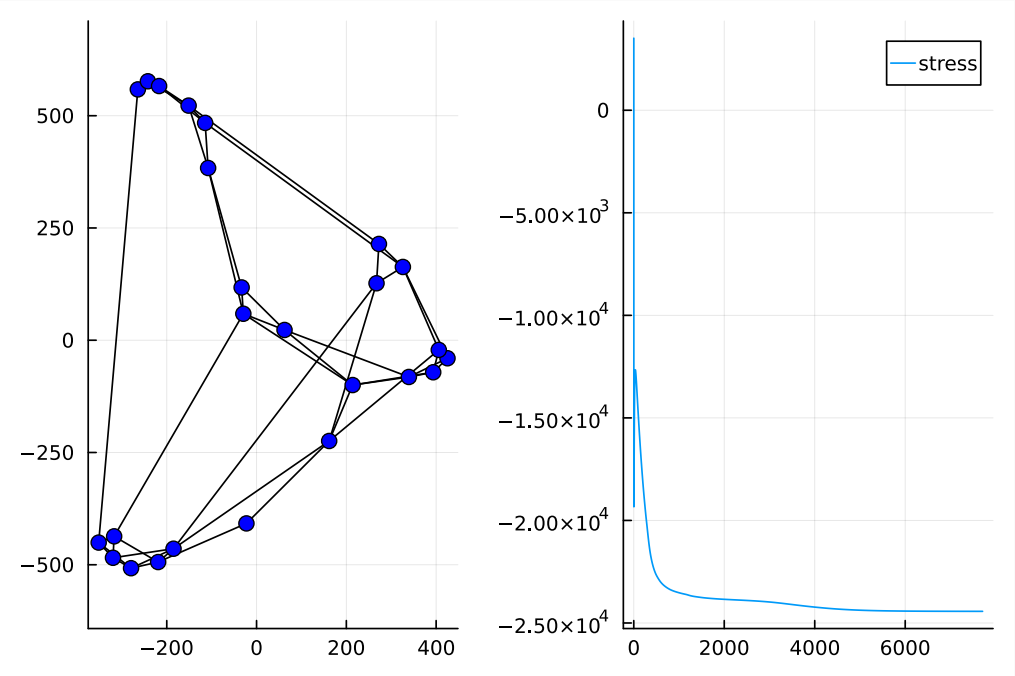

Figure 15: Watts Strogatz Graph, Took 7710 Iterations

Watts Strogatz Graph – FR

Nesterov Gradient Descent, FR, $\alpha = 0.001$, $\epsilon = 1e - 7$

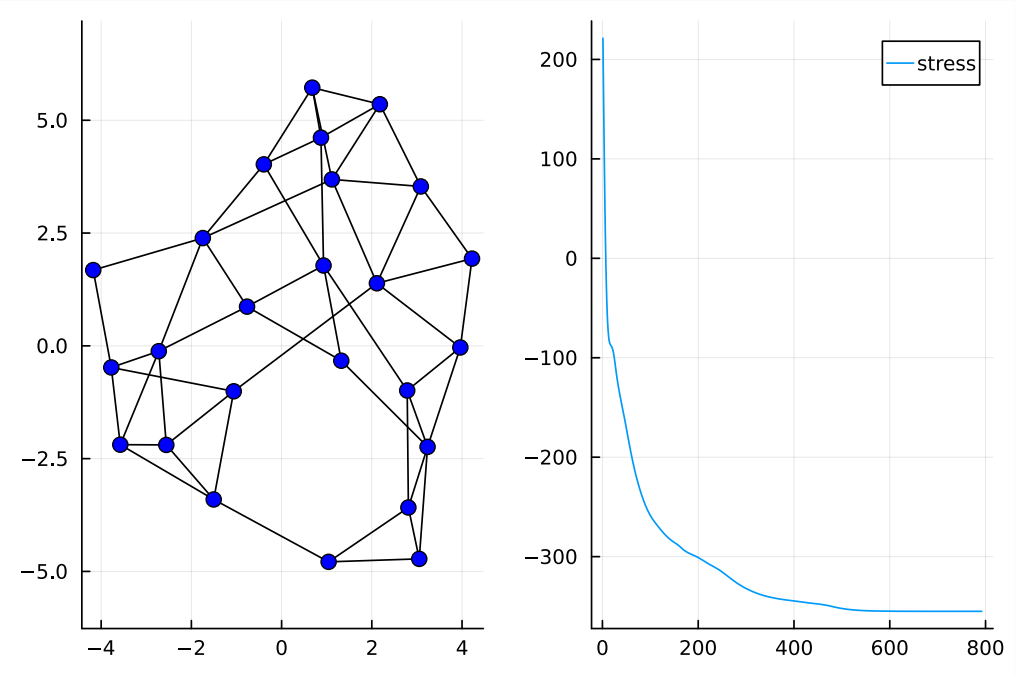

Figure 16: Watts Strogatz Graph, Took 791 Iterations

Static Scale Free Graph – ERLL

Nesterov Gradient Descent, ERLL, $\alpha = 0.1$, $\epsilon = 1e - 7$

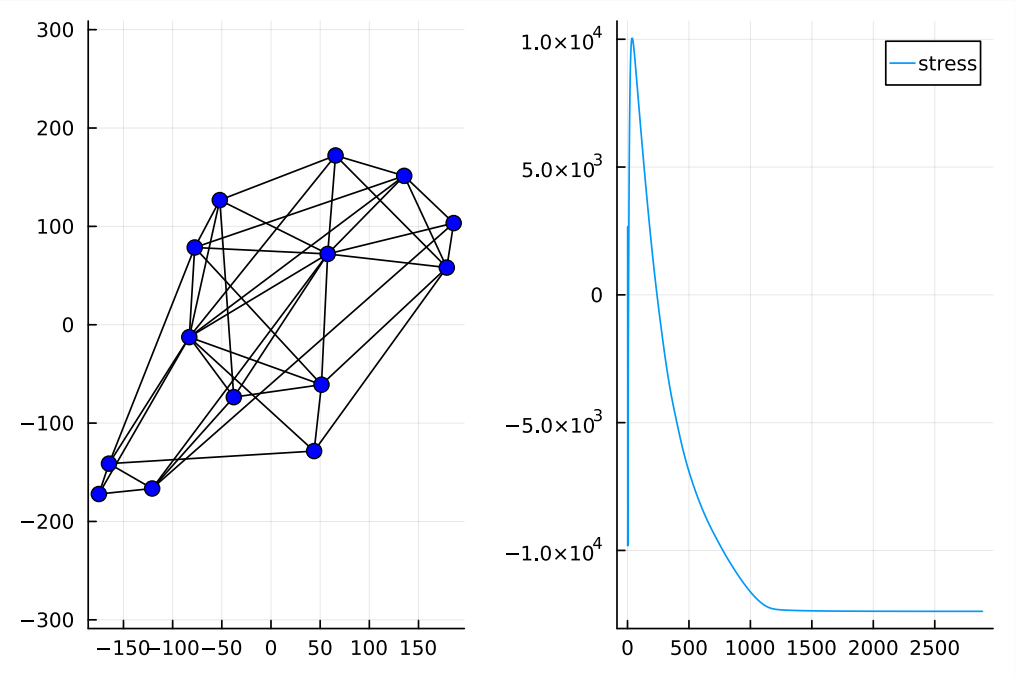

Figure 17: Static Scale Free Graph, Took 2887 Iterations

Static Scale Free Graph – FR

Nesterov Gradient Descent, FR, $\alpha = 0.001$, $\epsilon = 1e - 7$

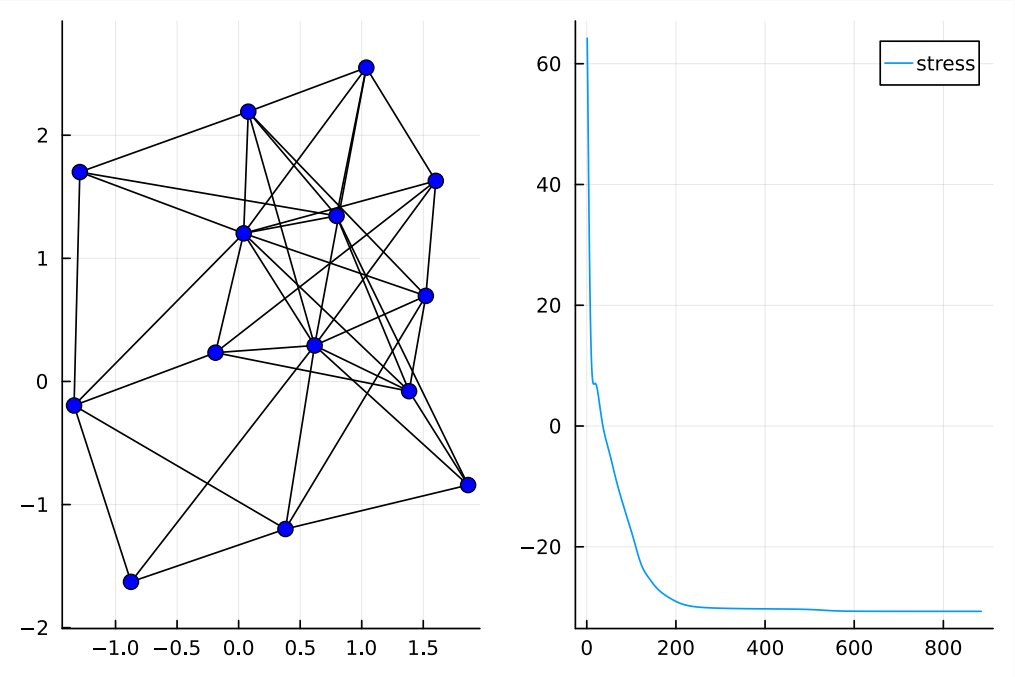

Figure 18: Static Scale Free Graph, Took 884 Iterations

Conclusion

- FR stress is much faster than ERLL stress
- FR stress produces better results than ERLL stress
- ERLL stress pushes vertices too far apart
- Some graphs get stuck in suboptimal positions, e.g. circular ladder graph
- Bipartite graphs get drawn in a very unusual way with this method

For a truly good graph drawing algorithm, many improvements are possible.

References i

[1] Andreas Noack. **"Energy Models for Drawing Clustered Small-World Graphs".** In: *Proceedings of International Symposium on Graph Drawing* (Aug. 2003).

[2] Dahua Lin and contributors. ***Distances.jl: A Julia Package for Evaluating Distances (Metrics) Between Vectors.*** Julia package, JuliaStats organization. 2014. URL: `https://github.com/JuliaStats/Distances.jl`.

[3] Michael Innes. **"Don't Unroll Adjoint: Differentiating SSA-Form Programs".** In: *CoRR* abs/1810.07951 (2018). arXiv: `1810.07951`. URL: `http://arxiv.org/abs/1810.07951`.

References ii

[4] Mirco Schönfeld and Jürgen Pfeffer. **"Fruchterman/Reingold (1991): Graph Drawing by Force-Directed Placement".** In: *Schlüsselwerke der Netzwerkforschung*. Ed. by Boris Holzer and Christian Stegbauer. Wiesbaden: Springer Fachmedien Wiesbaden, 2019, pp. 217–220. ISBN: 978-3-658-21742-6. DOI: 10.1007/978-3-658-21742-6_49. URL: https://doi.org/10.1007/978-3-658-21742-6_49.

[5] James Fairbanks et al. ***JuliaGraphs/Graphs.jl: an optimized graphs package for the Julia programming language.*** 2021. URL: https://github.com/JuliaGraphs/Graphs.jl/.

References iii

[6] Simon Christ et al. **"Plots.jl – A User Extendable Plotting API for the Julia Programming Language".** In: *Journal of Open Research Software* (2023). DOI: 10.5334/jors.431. URL: https://openresearchsoftware.metajnl.com/articles/10.5334/jors.431.